

Practical 5

17ce049

Aim -Use sklearn.datasets iris flower dataset to train your model using logistic regression. You need to figure out accuracy of your model and use that to predict different samples in your test dataset. In iris dataset there are 150 samples containing following features,

Sepal Length Sepal Width Petal Length Petal Width Using above 4 features you will classify a flower in one of the three categories,

Setosa Versicolour Virginica

In [1]:

```
from sklearn.datasets import load_iris
%matplotlib inline
import matplotlib.pyplot as plt
```

In [2]:

```
iris = load_iris()
```

In [3]:

```
dir(iris)
```

Out[3]:

```
['DESCR', 'data', 'feature_names', 'filename', 'target', 'target_names']
```

In [16]:

Out[16]:

```

'.. _iris_dataset:\n\nIris plants dataset\n-----\n\n**Data Set
Characteristics:**\n\n      :Number of Instances: 150 (50 in each of three c
lasses)\n      :Number of Attributes: 4 numeric, predictive attributes and the
class\n      :Attribute Information:\n          - sepal length in cm\n          -
sepal width in cm\n          - petal length in cm\n          - petal width in cm
\n          - class:\n              - Iris-Setosa\n              - Iris-Ve
rsicolour\n              - Iris-Virginica\n              \n      :Summary
Statistics:\n\n      =====\n      ==\n\n          Min  Max   Mean   SD   Class Correlation\n      =====\n      =====\n      sepal length:
4.3  7.9   5.84   0.83   0.7826\n      sepal width:   2.0  4.4   3.05   0.43
-0.4194\n      petal length:   1.0  6.9   3.76   1.76   0.9490 (high!)\n
petal width:   0.1  2.5   1.20   0.76   0.9565 (high!)\n      =====
= =====\n\n      :Missing Attribute Val
ues: None\n      :Class Distribution: 33.3% for each of 3 classes.\n      :Creat
or: R.A. Fisher\n      :Donor: Michael Marshall (MARSHALL%PLU@io.arc.nasa.gov)
\n      :Date: July, 1988\n\nThe famous Iris database, first used by Sir R.A.
Fisher. The dataset is taken\nfrom Fisher\'s paper. Note that it\'s the same
as in R, but not as in the UCI\nMachine Learning Repository, which has two w
rong data points.\n\nThis is perhaps the best known database to be found in
the\npattern recognition literature. Fisher\'s paper is a classic in the fi
eld and\nis referenced frequently to this day. (See Duda & Hart, for exampl
e.) The\ndata set contains 3 classes of 50 instances each, where each class
refers to a\ntype of iris plant. One class is linearly separable from the o
ther 2; the\nlatter are NOT linearly separable from each other.\n\n.. topi
c:: References\n\n      - Fisher, R.A. "The use of multiple measurements in tax
onomic problems"\n          Annual Eugenics, 7, Part II, 179-188 (1936); also in
"Contributions to\n          Mathematical Statistics" (John Wiley, NY, 1950).\n
- Duda, R.O., & Hart, P.E. (1973) Pattern Classification and Scene Analysi
s.\n          (Q327.D83) John Wiley & Sons. ISBN 0-471-22361-1. See page 218.\n
- Dasarthy, B.V. (1980) "Nosing Around the Neighborhood: A New System\n
Structure and Classification Rule for Recognition in Partially Exposed\n
Environments". IEEE Transactions on Pattern Analysis and Machine\n          Inte
lligence, Vol. PAMI-2, No. 1, 67-71.\n      - Gates, G.W. (1972) "The Reduced N
earest Neighbor Rule". IEEE Transactions\n          on Information Theory, May 1
972, 431-433.\n      - See also: 1988 MLC Proceedings, 54-64. Cheeseman et a
l\'s AUTOCLASS II\n          conceptual clustering system finds 3 classes in the d
ata.\n      - Many, many more ...'

```

In [14]:

iris.target_names

Out[14]:

array(['setosa', 'versicolor', 'virginica'], dtype='<U10')

In [15]:

```
iris.feature_names
```

Out[15]:

```
['sepal length (cm)',  
'sepal width (cm)',  
'petal length (cm)',  
'petal width (cm)']
```

In [4]:

```
iris.data[0]
```

Out[4]:

```
array([5.1, 3.5, 1.4, 0.2])
```

In [5]:

```
from sklearn.linear_model import LogisticRegression  
model = LogisticRegression()
```

In [6]:

```
from sklearn.model_selection import train_test_split  
X_train, X_test, y_train, y_test = train_test_split(iris.data, iris.target, test_size=0.2)
```

In [7]:

```
model.fit(X_train, y_train)
```

Out[7]:

```
LogisticRegression(C=1.0, class_weight=None, dual=False, fit_intercept=True,  
                    intercept_scaling=1, l1_ratio=None, max_iter=100,  
                    multi_class='auto', n_jobs=None, penalty='l2',  
                    random_state=None, solver='lbfgs', tol=0.0001, verbose=0,  
                    warm_start=False)
```

In [8]:

```
model.score(X_test, y_test)
```

Out[8]:

```
0.9666666666666667
```

In [27]:

```
def predict_flower(x):  
    prediction = model.predict(x.reshape(1, -1))  
    return iris.target_names[prediction][0]
```

In [37]:

```
print(f'flower with features {X_test[0]} is predicted {predict_flower(X_test[0])} and origi
print(f'flower with features {X_test[5]} is predicted {predict_flower(X_test[5])} and origi
print(f'flower with features {X_test[10]} is predicted {predict_flower(X_test[10])} and ori
print(f'flower with features {X_test[15]} is predicted {predict_flower(X_test[15])} and ori
print(f'flower with features {X_test[20]} is predicted {predict_flower(X_test[20])} and ori
```

```
flower with features [5.3 3.7 1.5 0.2] is predicted setosa and originally it
is setosa
flower with features [6.  3.4 4.5 1.6] is predicted versicolor and originall
y it is versicolor
flower with features [7.  3.2 4.7 1.4] is predicted versicolor and originall
y it is versicolor
flower with features [6.4 2.8 5.6 2.1] is predicted virginica and originally
it is virginica
flower with features [7.7 3.  6.1 2.3] is predicted virginica and originally
it is virginica
```

In [10]:

```
y_predicted = model.predict(X_test)
y_predicted
```

Out[10]:

```
array([0, 1, 2, 2, 0, 1, 1, 2, 1, 1, 1, 1, 1, 2, 1, 2, 0, 2, 2, 2, 0,
       2, 1, 1, 2, 0, 2, 1, 0])
```

In [11]:

```
iris['target_names']
```

Out[11]:

```
array(['setosa', 'versicolor', 'virginica'], dtype='<U10')
```

In [12]:

```
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test, y_predicted)
cm
```

Out[12]:

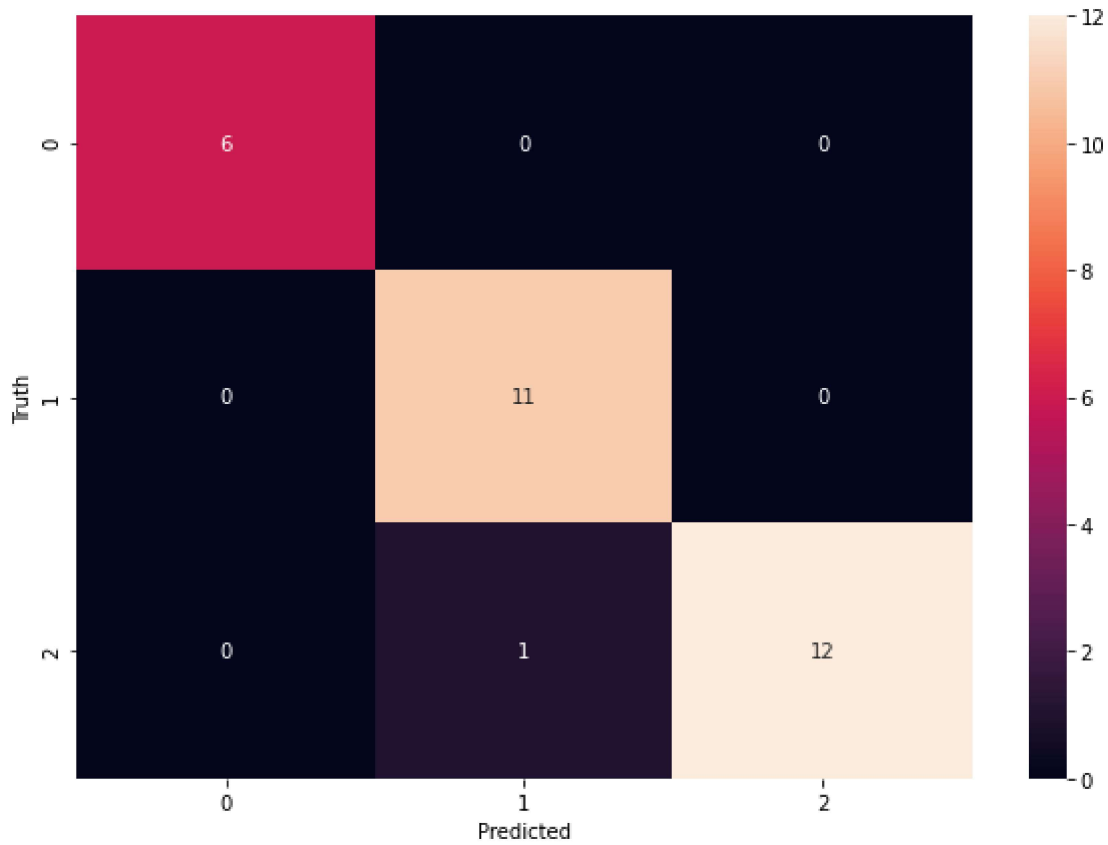
```
array([[ 6,  0,  0],
       [ 0, 11,  0],
       [ 0,  1, 12]], dtype=int64)
```

In [13]:

```
import seaborn as sn
plt.figure(figsize = (10,7))
sn.heatmap(cm, annot=True)
plt.xlabel('Predicted')
plt.ylabel('Truth')
```

Out[13]:

Text(69.0, 0.5, 'Truth')



In []: